

MGMT 803, MBA@RICE, 2026

Thursday, September 24, Afternoon

Kerry Back

J. Howard Creekmore Professor of Finance

This afternoon

Permissions

What to allow without approval

Persistent instructions

System prompts, CLAUDE.md/AGENTS.md, SKILL.md, memories

Connectors and plugins

Tools on somebody else's server, and folders of skills other people published

7 · PERMISSIONS

What Claude asks before it acts

MODE	WHAT IT DOES
Manual	asks before the first use of each tool
Plan	reads and explores, edits nothing
Accept edits	file edits go through without asking
Auto	approves on its own, with a check on each action
Don't ask	refuses anything it would otherwise ask about
Bypass	no prompts at all

Enforced by the program around the model, not by the model. A line in `CLAUDE.md` shapes what Claude tries to do; it does not change what is allowed.

Permissions in Claude Desktop

The desktop app runs Claude Code, so the modes are the same and carry the same names.

What it can reach

the folders you connect to the conversation, and nothing else

What it asks about

the first use of each tool, in Manual mode

What “don’t ask again” does

writes a rule that outlives the conversation

Granting a folder is the decision that matters. The mode decides how often you are asked; the folder decides what can be reached at all.

Everything is allowed already, in `~/.claude/settings.json`:

```
1 "permissions": {  
2   "allow": ["Bash(*)", "Read(*)", "Write(*)", "Edit(*)",  
3           "Glob(*)", "Grep(*)", "WebFetch(*)", "WebSearch(*)"],  
4   "deny": []  
5 }
```

So nothing prompts you all week. That is a decision about this container — it holds your course files and is reachable only by you — not about how to run an agent on your own machine.

Permissions in your container

8 · PERSISTENT INSTRUCTIONS

Two kinds of instruction

With no instructions of your own, a model answers everyone's question the same way. Instructions are how that gets changed.

The provider's

The **system prompt** ships with the product and is sent on every turn. It regulates behavior: what the assistant is, what it declines, how an answer is formatted. You do not edit it.

Yours

Added on top. It customizes behavior: how you want work done, what your files hold, the conventions you follow. You choose what it says and where it applies.

What reaches the model before your prompt

PART	WHO SETS IT	WHEN IT IS SENT
System prompt	the provider	every turn
Tool definitions	the product, and what you connect	every turn
CLAUDE.md	you	every turn
A skill's description	you	every turn, one line each
A skill's body	you	only when the description matches

Anything sent on every turn costs money and latency, and takes up the model's attention.

What is in a system prompt

Identity

what it is, who made it, what today's date is

Format

when to use markdown, how long an answer should run, when to use a list

Tools

when to search the web, when to write code, when to make an artifact

Refusals

what it declines, and how

Character

what to do when it is uncertain, how much to hedge

Anthropic publishes the ones in the Claude apps, dated and by model:
platform.claude.com/docs/en/release-notes/system-prompts

CLAUDE.md, by scope

Claude Code reads every `CLAUDE.md` from the top of the filesystem down to the folder you started in, and puts them all in context:

FILE	APPLIES TO	
<code>\$HOME/.claude/CLAUDE.md</code>	every folder you work in	nobody
<code>path/folder/CLAUDE.md</code>	work in that folder	the team, if you commit it

The closest file is read last. Keep each one under about 200 lines — all of them are in the context window before you type.

- OpenAI and Google call it `AGENTS.me`
- Desktop apps have analogous “instructions” settings

A digression on file systems

Windows File Explorer hides the actual filesystem locations of common folders. You can do the following to see where your home folder is.

- Windows: Start -> Command Prompt. Then type `cd $HOME` and Enter and then `pwd` and Enter. That is your home directory.
- Mac: Enter the same commands into the Terminal app.
- lab803: Terminal in left pane. Enter the same commands.

Everything above is written by you. Claude writes this one, as you chat, and puts it in front of itself the next time you open a conversation.

- 1 While you talk, it saves individual topics — not a transcript, and not a summary written when the conversation ends.
- 2 Those topics arrive with the system prompt, ahead of your first message.
- 3 Nothing is searched and no tool runs. There is no sign of it in the conversation.
- 4 Each project has its own memory. Topics from one do not reach another.

Memory

Pushed, or pulled

MEMORY

Saved as you go and placed in the conversation without your asking. Whether it applies is decided before you type.

CHAT SEARCH

You ask for it. It searches your past conversations, you see the tool run, and the answer cites the chat each piece came from.

Settings → Memory lists the topics. Read, edit or delete any of them, pause it, or reset it. Say what to remember or forget in the chat and it applies to your next conversation. An incognito chat forms no memory.

Yes

- How you want numbers handled
- What the folders hold
- Conventions everything here follows
- Anything you have corrected twice

No

- Steps for one particular task
- Anything longer than a page
- Background that can be looked up
- A tool you use once a month

In Claude Code, `#` at the start of a message appends a line to one of these files, and `/memory` opens them for editing.

What belongs in a CLAUDE.md

Explore

- In lab803, enter this prompt: `print the CLAUDE.md`
- Then this prompt: `add "always answer like a pirate" to the CLAUDE.md`
- Then log out (top right corner), log back in, and ask anything.
- Then ask it to delete the “always answer like a pirate” from the CLAUDE.md, log out again, and log back in.

SKILLS

A skill is a folder

<code>skill-name/</code>	
<code>SKILL.md</code>	the instructions
<code>references/</code>	longer material, read only when needed
<code>scripts/</code>	code the skill runs rather than regenerates

`SKILL.md` opens with two lines of metadata (name and description) and then plain text.

In Claude Desktop: Choose Customize in left pane. Then Skills. Explore or Add.

The same file in other agents

Anthropic wrote the format and released it as an open standard. The others adopted it rather than inventing their own.

AGENT	WHERE IT LOOKS
Claude Code	<code>.claude/skills/</code>
Codex and ChatGPT	<code>.agents/skills/</code>
Gemini CLI	<code>.gemini/skills/</code> , or <code>.agents/skills/</code>

Same file, same description-first loading. Put the folder in `.agents/skills/` and both Codex and Gemini CLI find it.

Creating a skill

- Work with AI until you get what you want
- Tell AI to create a skill so you can get the same thing the next time
- Give the skill a meaningful name.
- As you use it and find things to add or modify, tell AI to edit the skill.

Three ways to invoke

Slash command	<code>/skill-name</code>
By name	“use the ... skill”
Automatically	“build ...”

The third only works when the skill description clearly applies.

Anthropic's skill repository

<https://github.com/anthropics/skills>

lab803

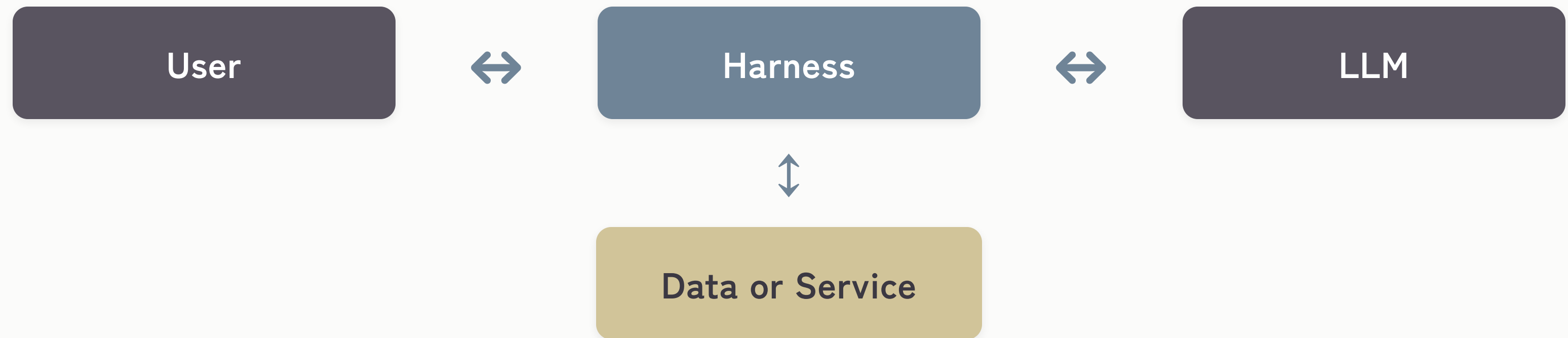
- Enter the prompt `/skills`
- You will get a list of the installed skills
- All are from Anthropic

Breakout exercise

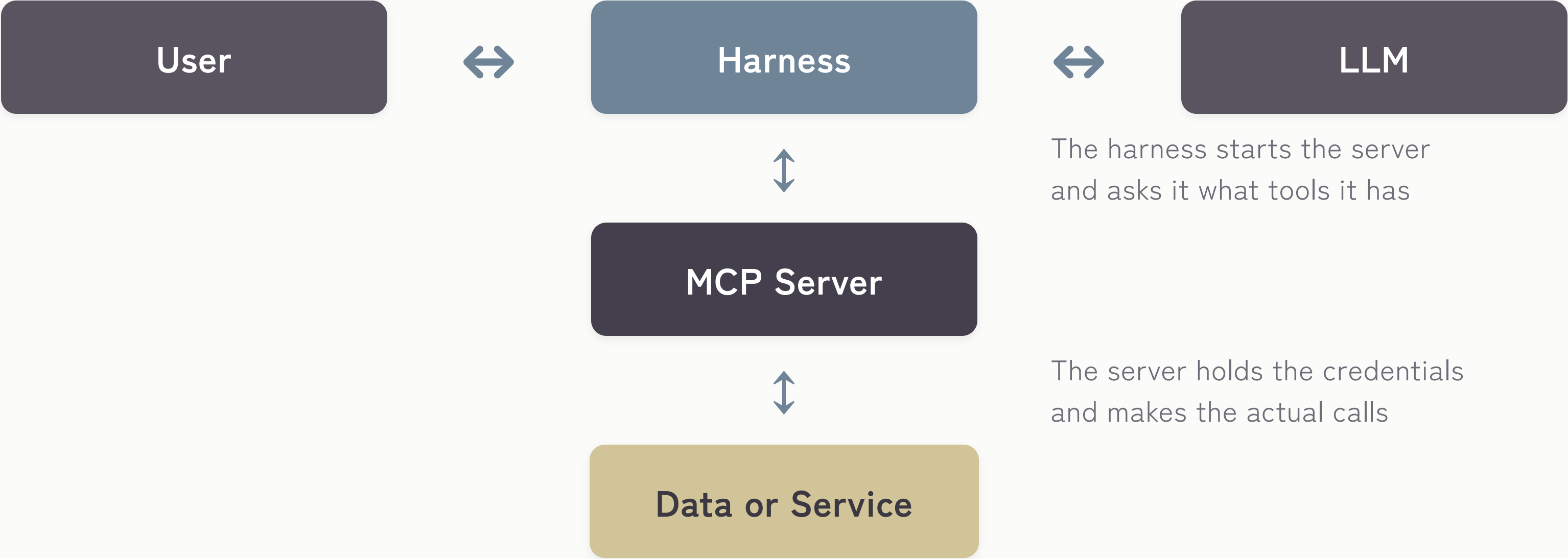
- The goal is to get Claude to always create PowerPoint decks following a template you choose. You can use one of your existing decks or choose a new template from any online source. Upload the deck to lab803.
- Tell Claude to create a new deck using the style of the uploaded deck. Give Claude some content and tell it to build the deck from scratch rather than just editing the uploaded deck.
- Download and view the deck. Instruct Claude on any changes you want. Rinse and repeat until you like the deck.
- Tell Claude to write the steps needed to get the good deck in a my-pptx skill so that Claude will always create decks in the template style.

9 · CONNECTORS AND PLUGINS

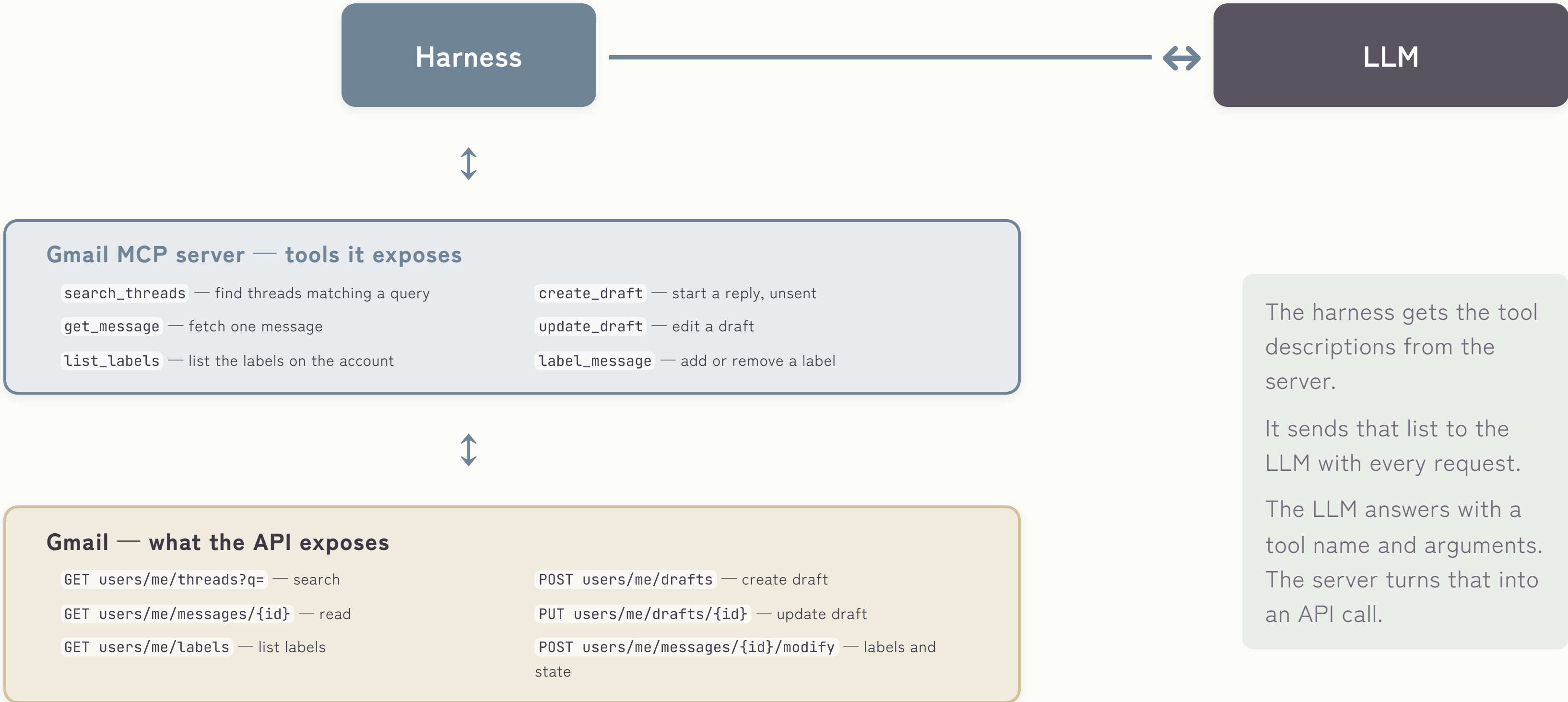
An agent reaching a service



An MCP server is a bridge



What the server tells the harness



Tools from MCP servers

Tools written by somebody else, running on a server somebody else operates. Thousands of MCP servers exist already.

What you add

A URL, in the agent's configuration

What arrives

A name, a description and an input schema for every tool the server publishes

What stays there

The code, and the credential it uses to reach the service

To the model these are more entries in the same tool list. What differs is whose machine runs them.

Example: Google's Gmail server

search_threads	get_thread	get_message
create_draft	list_drafts	list_labels
label_thread	unlabel_thread	
label_message	unlabel_message	

Ten tools, at <https://gmailmcp.googleapis.com/mcp/v1>. Published at developers.google.com/workspace/gmail/api/reference/mcp.

Example: Anthropic's Gmail server

	GOOGLE	ANTHROPIC
Tools	10	29
Read	<code>search_threads</code> · <code>get_thread</code> · <code>get_message</code>	the same three
Drafts	<code>create_draft</code>	create, get, update, delete
Send	none	<code>send_message</code> · <code>reply</code> · <code>forward</code>
Remove	none	<code>trash_message</code> · <code>delete_draft</code>
Labels	apply and remove	plus create, update, delete

The same account and the same API underneath. A different answer to what an agent may do with it.

Warehouses

Snowflake, Databricks, BigQuery,
MongoDB, dbt

Business systems

Salesforce, HubSpot, Workday, SAP,
ServiceNow

Work and code

Slack, Notion, Linear, GitHub, Atlassian,
Figma, Sentry

Money

Stripe, Shopify, PayPal, Plaid

Example MCP servers

Connecting to an MCP server

In Claude Desktop

- Choose Customize from the left pane
- Then Connectors and Explore or Add

In lab803

- Tell Claude to add the connection
- You provide the URL, or Claude can search for it online
- Claude may say it can't do it directly but then ask permission to use a terminal command. Allow.
- Log out and log in again.

Example: `fred.kerryback.com/mcp`, the Federal Reserve's economic data.

Ask Claude to get the US consumer price index from FRED, compute quarterly percent changes, and annualize them.

Exercise

Breakout exercise

Connect to a Gmail MCP server either in a desktop app or on lab803. Create a new Gmail account if you don't want to connect to your real one. Ask Claude to help connect. You can use any server (Anthropic, Google, ...). Ask Claude how to disable any tools you don't want. Then ask Claude to get your most recent messages. Ask Claude how to delete the connection if you don't want to keep it. Ask Claude to write a summary of what was done.

INSTALLING PLUGINS

What you install

A plugin is a folder somebody else published. Installing it adds their work to your Claude in one step.

What it can carry

Skills, slash commands, subagents, hooks, and MCP servers.

What you get

Every file and script in it, without being told which ones to copy where.

Anthropic's plugin marketplace

<https://code.claude.com/docs/en/discover-plugins>

Exercise

Give Claude the following prompts

```
/plugin marketplace add kerryback/skills  
/plugin install finance-data@kerryback
```

Then ask Claude what was installed. Finally, ask Claude to compute monthly stock returns for Apple since the start of 2020 and to produce a histogram.

You should see Claude say “using the finance-data skill”